Main Activity – Layout Description

Overview

The main activity of this Android application serves as the central dashboard for reading, displaying, and controlling data from all registered sensors. Currently registered are: an accelerometer, a gyroscope, and a magnetometer. The screen is organized as a single vertical `LinearLayout` that fills the entire display and is divided into five distinct visual zones: a header bar, an optional error card, a scrollable sensor data content area, a status and control section in the lower portion, and a footer navigation bar. The background color of the root layout is defined by the app theme color `@color/background`, giving the screen a consistent base appearance. The activity is associated with the class `MainActivity` located in the `main` package.

Header Bar

At the top of the screen sits a horizontal `LinearLayout` styled with the `@color/header` background. This bar has a fixed height determined by its content and spans the full screen width. It contains three elements arranged side by side using layout weights to achieve a balanced distribution.

On the left side is an `ImageButton` (`settings_button`) displaying a settings gear icon (`outline_settings_24`). It has a top margin of 16 dp and a start margin of 16 dp, with a bottom padding of 15 dp to align it visually within the bar. Its icon color matches the app's text color. Tapping this button is expected to open a settings screen or dialog where the user can configure application parameters such as MQTT connection details or sensor preferences.

In the center, a `TextView` (`headerText`) displays the app's main title. The text is drawn from the string resource `@string/main_header` and is rendered in bold at 28 sp, centered both horizontally and vertically within a fixed height of 80 dp. This acts as the application's branding element.

On the right side is a second `ImageButton` (`notification_button`) showing a notification bell icon (`outline_circle_notifications_24`), mirroring the settings button's margins on the opposite side. Tapping it is intended to open a notifications panel or alert history view.

MQTT Error Card

Directly below the header, a `CardView` (`card_mqtt_error`) is defined with rounded corners (12 dp radius) and a subtle elevation of 4 dp. By default, its visibility is set to `gone`, meaning it

is hidden unless programmatically made visible at runtime. It becomes visible when the application fails to establish a connection to an MQTT broker.

Inside the card, a vertical `LinearLayout` with `@color/header` background contains two horizontally arranged sub-layouts. The first displays an icon (`outline_android_wifi_4_bar_off_24`) that visually represents a loss of wireless connectivity. The second contains a `TextView` showing the error message: "Es konnte keine Verbindung zu MQTT aufgebaut werden!", rendered at 20 sp. This card acts as a non-intrusive inline warning that the telemetry pipeline is currently unavailable.

Sensor Data Content Area

The main content region is a vertical `LinearLayout` (`contentArea`) that uses `layout_weight="1"` to fill all remaining vertical space not occupied by the header and footer. It has 16 dp of padding on all sides and contains three sensor data blocks, each separated by a thin horizontal divider line (1 dp height, `@color/button` background).

****Accelerometer Block:**** The first block is headed by a bold `TextView` (`accelTitle`) at 20 sp, populated from `@string/sensor_accel_title`. Beneath it are three `TextViews` – `accelXValue`, `accelYValue`, and `accelZValue` – displayed at a large 28 sp size in the `@color/highlight` color and centered horizontally. These fields are populated at runtime with live accelerometer readings for the X, Y, and Z axes respectively. Their initial values are set from string resources `@string/beschleunigung_x`, `@string/beschleunigung_y`, and `@string/beschleunigung_z`.

****Gyroscope Block:**** Separated from the accelerometer block by a divider, the gyroscope section follows the same structure. A bold title `TextView` (`gyroTitle`) is drawn from `@string/sensor_gyro_title`, and three value `TextViews` (`gyroXValue`, `gyroYValue`, `gyroZValue`) display the angular velocity for each axis, initialized from `@string/gyro_x`, `@string/gyro_y`, and `@string/gyro_z`.

****Magnetometer Block:**** The third and final sensor block, also preceded by a divider, displays magnetic field data. Its title (`magTitle`) comes from `@string/sensor_magnet_title`, and the three axis values (`magXValue`, `magYValue`, `magZValue`) are initialized from `@string/magnet_x`, `@string/magnet_y`, and `@string/magnet_z`. These represent the magnetic field strength along each spatial axis.

At the bottom of the content area, a `Button` (`graphenansicht`) is displayed with a top margin of 30 dp and is centered horizontally. Its label comes from `@string/btn_graph`. Tapping this button navigates the user to a graphical visualization view where sensor data can be observed live as graphs.

Status Labels and Transmission / Operating Mode Controls

Below the content area, separated by a full-width divider, is a vertical status block containing two centered `TextViews`. `ModeLabel` (id) displays the current transmission mode, initialized with `@string/modus_default`. `OperatingModeLabel` displays the current operating mode, initialized with `@string/betriebsmodus_default`. Both labels are rendered in 14 sp in the theme text color. These labels update dynamically to reflect the currently active mode selections made via the buttons below.

Following another divider, a horizontal row of three equally weighted `Buttons` allows the user to select the data transmission mode:

- **Stream** (`btnStream`): Causes sensor data to be transmitted continuously in real time. This button is rendered at full opacity (1.0), indicating it is the currently active mode by default.

- **Burst** (`btnBurst`): Causes the device to send a rapid batch of readings in a short window. Displayed at reduced opacity (0.4), indicating it is currently inactive.

- **Average** (`btnAverage`): Causes the device to transmit averaged sensor values over a defined interval. Also displayed at reduced opacity (0.4).

Below that row, a second horizontal row of four equally weighted `Buttons` controls the operating mode of the connected device or system:

- **Autark** (`btnAutark`): The device operates independently without external supervision. Shown at 0.4 opacity, currently inactive.

- **Supervision** (`btnSupervision`): The device operates under remote monitoring. Shown at full opacity (1.0); the currently active operating mode.

- **Event** (`btnEvent`): The device only transmits data when a specific trigger event occurs. Shown at 0.4 opacity, currently inactive.

- **Identif.** (`btnIdentification`): Identification mode. Shown at 0.4 opacity, currently inactive.

All transmission and operating mode buttons share the `@color/button` background tint (for the transmission row) or `@color/header` tint (for the operating mode row), with white text drawn from `@color/background`. Active buttons are visually distinguished from inactive ones through opacity: a value of 1.0 marks the currently selected option, while 0.4 marks the others. When the user taps a button, the app is expected to update the MQTT control topic accordingly and refresh both the button opacities and the status labels in the middle section.

Footer Navigation Bar

The bottom of the screen is occupied by a fixed-height (80 dp) horizontal `LinearLayout` (`footerBar`) styled with `@color/header` as the background. It contains four equally weighted controls:

- **Accel button** (`btnAccel`): Navigates the content area to the accelerometer view. The label is sourced from `@string/btn_accel`.

- **Gyro button** (`btnGyro`): Navigates to the gyroscope data view. Label from `@string/btn_gyro`.

- **Magnet button** (`btnMagnet`): Navigates to the magnetometer view. Label from `@string/btn_magnet`.

- **Voice button** (`btnVoice`): An `ImageButton` displaying the Android system microphone icon (`ic_btn_speak_now`). It triggers Android's built-in `SpeechRecognizer`, allowing the user to issue voice commands to the application. It has 12 dp of internal padding and is described with the content description "Sprachbefehl" (German for "voice command").

The first three footer buttons share the `@color/button` background tint and the app's background color for text. They act as quick-navigation shortcuts that scroll or switch the content area to the relevant sensor section, enabling fast access on smaller screens where all three sensor blocks may not be simultaneously visible.

 # Events Activity (Ereignisse) – Layout Description

Overview

The Events activity ("Ereignisse") provides a structured log of recorded sensor events. It is a secondary screen within the application, reachable from the main activity, and presents event data in a sortable, scrollable list. The root layout is a vertical `LinearLayout` (`ereignisse`) that fills the entire screen, using `@color/background` as the base color. The screen is composed of three main zones: a header bar at the top, a column header bar with sort controls beneath it, a scrollable event list occupying the bulk of the screen, and a floating action button anchored to the bottom right corner.

Header Bar

The topmost element is a horizontal `LinearLayout` styled with `@color/header` as its background, spanning the full screen width. It follows the same visual structure as the main activity's header and contains three elements.

On the left sits an `ImageButton` (`home_button`) displaying a home icon (`baseline_home_24`). It carries the content description "Zur Startseite" (German for "To the start page"), making its purpose clear: tapping it navigates the user back to the main activity. Its margins and padding match the header buttons of the main screen, ensuring visual consistency across the app.

In the center, a bold `TextView` (`headerText`) displays the screen title sourced from `@string/ereignisse_title`, rendered at 26 sp with center alignment both horizontally and vertically within an 80 dp tall field. This title identifies the screen to the user as the events log.

On the right is a placeholder `ImageButton` with no ID and no icon source assigned, acting as a visual counterweight to the home button on the left. It maintains the symmetrical three-column layout of the header bar but currently carries no tap action.

Sort Header Bar

Directly below the header, a horizontal `LinearLayout` (`sort_header_bar`) serves as a column label row for the event list beneath it. It has horizontal padding of 16 dp, vertical padding of 8 dp, a small bottom and top margin of 5 dp, and a slight elevation of 2 dp to visually lift it above the list content. Its background is `@color/button`, clearly distinguishing it from the main background and making the column labels stand out.

The bar contains four equally weighted `TextViews`, each occupying one quarter of the available width via `layout_weight="1"`. Each label is displayed in bold at 12 sp in the app's background color, creating a high-contrast appearance against the button-colored bar. All four labels respond to touch via `?attr/selectableItemBackground`, meaning they produce a visual ripple effect when tapped, indicating they are interactive sort controls:

- **TYP** (`btn_sort_type`): Sorts the event list by sensor type (e.g., accelerometer, gyroscope, magnetometer).
- **EREIGNISTYP** (`btn_sort_sensor`): Sorts by the type of event that was recorded (e.g., threshold breach, peak detection).
- **WERT** (`btn_sort_value`): Sorts the list by the numerical value associated with the event.
- **ZEIT** (`btn_sort_timestamp`): Sorts the list by the timestamp of when the event occurred.

Tapping any of these column headers is expected to re-sort the list displayed in the `RecyclerView` below, either in ascending or descending order depending on the implementation logic in the activity class.

Event List

The primary content area is a `RecyclerView` (`my_table_recyclerview`) that fills all remaining vertical space between the sort header bar and the bottom of the screen, achieved through `layout_height="0dp"` combined with `layout_weight="1"`. It spans the full screen width with no additional padding defined at the layout level.

The `RecyclerView` renders the list of recorded sensor events, where each row corresponds to one event entry and is expected to display data aligned with the four column headers above: the sensor type, the event type, the measured value, and the timestamp. The list is scrollable, accommodating an arbitrary number of event records without truncating the display. The actual row layout and data binding are defined in a separate item layout file and adapter class not contained in this file.

Add Event Button

In the bottom-right corner of the screen, an `ImageButton` (`new_event`) is positioned using `layout_gravity="end"`. It displays a circular add icon (`outline_add_circle_24`) and carries the content description "Zur Startseite", though functionally it is intended to open a form or dialog for manually creating a new event entry. It has a right margin of 16 dp and a bottom padding of 16 dp, keeping it visually clear of the screen edge. Its background matches `@color/background`,

blending with the screen while keeping the icon itself prominent. Tapping this button is expected to trigger an input flow – such as a dialog or a new activity – where the user can define and save a new event record to the list.

 # Accelerometer Activity (Beschleunigungssensor) – Layout Description

Overview

The Accelerometer activity ("Beschleunigungssensor") is a dedicated detail screen for visualizing accelerometer data as interactive time-series charts. It is one of three sensor-specific chart screens in the application, alongside equivalent screens for the gyroscope and magnetometer. The root layout is a vertical `LinearLayout` (`accel`) that fills the entire display, using `@color/background` as the base color. The screen is divided into three zones: a header bar at the top, a main content area containing the time range controls, three line charts, and axis visibility checkboxes, and a footer navigation bar at the bottom.

Header Bar

The header follows the same visual pattern as the other screens in the application. It is a horizontal `LinearLayout` with a `@color/header` background and contains three elements.

On the left, an `ImageButton` (`home_button`) displays the home icon (`baseline_home_24`) with the content description "Zur Startseite", indicating that tapping it returns the user to the main activity. On the right, an `ImageButton` (`notification_button`) displays the notification bell icon (`outline_circle_notifications_24`), consistent with the main activity header. In the center, a bold `TextView` (`headerText`) shows the hardcoded title "Beschleunigungssensor" at 22 sp, centered both vertically and horizontally within an 80 dp tall field. The slightly smaller font size compared to the main activity header accommodates the longer German title without truncation.

Time Range Controls

Directly below the header, the main content area begins with a horizontal row of controls that allow the user to define the time window displayed across all three charts. The row contains the following elements arranged left to right:

A static label "Von: " (German for "From:") is followed by a compact `Button` (`button_x_von`) displaying a date and time string, defaulting to "01.01.2025 00:00". Tapping this button is expected to open a date-time picker dialog, allowing the user to set the start boundary of the displayed time range. Next, a label "Bis: " (German for "To:") with a 10 dp start margin is followed by a matching `Button` (`button_x_bis`) for the end of the time range, also defaulting to "01.01.2025 00:00". Both date buttons are 30 dp tall, styled with `@color/button` as the background tint and the app's background color for text, and use zero padding to keep them compact.

To the right of the date buttons, an `ImageButton` (`resetX`) displays a reset icon (`outline_reset_focus_24`) and is expected to restore the chart's time range to its default or full extent when tapped. Further to the right, with a 60 dp start margin creating a clear visual separation, a compact `Button` (`btn_x_10min`) labeled "10 Min" and styled with `@color/header` as the background tint provides a quick shortcut to zoom the chart view to the most recent ten minutes of data. This allows the user to instantly focus on recent readings without manually adjusting the date pickers.

Line Charts

The bulk of the content area is occupied by three `LineChart` views from the MPAndroidChart library, stacked vertically and each assigned equal height through `layout_weight="1"` within a vertical `LinearLayout` that itself uses `layout_weight="1"` to consume all remaining screen space between the header and footer.

- ****lineChartAccelX****: Displays accelerometer measurements along the X axis over the selected time range.

- ****lineChartAccelY****: Displays accelerometer measurements along the Y axis.

- ****lineChartAccelZ****: Displays accelerometer measurements along the Z axis.

Each chart has 8 dp of margin on all sides for breathing room. All three charts respond to the same time range defined by the "Von" and "Bis" controls above, meaning adjusting the time window or tapping "10 Min" updates all three charts simultaneously. The charts support standard MPAndroidChart interactions such as pinch-to-zoom, drag to scroll along the time axis, and tap to inspect individual data points.

Axis Visibility Checkboxes

Below the three charts, a horizontal row of three `CheckBox` controls allows the user to toggle the visibility of each axis chart independently:

- ****cbChartX**** – labeled "X axis", checked by default.

- ****cbChartY**** – labeled "Y axis", checked by default.

- ****cbChartZ**** – labeled "Z axis", checked by default.

All three are equally weighted and displayed in `@color/text`. When a checkbox is unchecked, the corresponding `LineChart` is expected to be hidden or its data series removed, giving the user a cleaner view when only one or two axes are of interest. By default all three are visible and active.

Footer Navigation Bar

The bottom of the screen is a fixed-height (80 dp) horizontal `LinearLayout` (`footerBar`) with `@color/header` background, providing navigation to the adjacent sensor chart screens. It contains four equally weighted elements:

On the far left, a `TextView` (`MagnetDescription`) displays "Mag" at 20 sp, indicating that the previous screen in the sensor sequence is the Magnetometer activity. Next to it, a `Button` (`btnPrevMagnet`) labeled with a left arrow ("<") navigates the user to the Magnetometer chart screen when tapped. On the right side, a `Button` (`btnNextGyro`) labeled with a right arrow (">") navigates to the Gyroscope chart screen. Finally, a `TextView` (`NextGyro`) displays "Gyro" at 20 sp, labeling the next screen in the sequence.

This footer establishes a left-to-right navigation order of Magnetometer → Accelerometer → Gyroscope, allowing the user to swipe through sensor detail screens using the arrow buttons without returning to the main activity each time.

 # Gyroscope Activity (Gyroskop) – Layout Description

Overview

The Gyroscope activity ("Gyroskop") is the dedicated detail screen for visualizing gyroscope sensor data as interactive time-series charts. It is structurally identical to the Accelerometer activity and forms the middle screen in the three-screen sensor chart navigation sequence: Accelerometer → Gyroscope → Magnetometer. The root layout is a vertical `LinearLayout` (`gyro`) that fills the entire display against a `@color/background` base. The screen is organized into three zones: a header bar, a main content area with time range controls, three axis charts, and axis visibility toggles, followed by a footer navigation bar.

Header Bar

The header is a horizontal `LinearLayout` with `@color/header` background, matching the visual style of all other screens in the application. It contains three elements in the same weighted arrangement used throughout the app.

On the left, an `ImageButton` (`home_button`) shows the home icon (`baseline_home_24`) with the content description "Zur Startseite". Tapping it navigates the user back to the main activity. On the right, an `ImageButton` (`notification_button`) displays the notification bell icon (`outline_circle_notifications_24`), consistent with the main activity and the Accelerometer screen. In the center, a bold `TextView` (`headerText`) displays the hardcoded title "Gyroskop" at 28 sp — the largest font of the three sensor detail screens — centered within an 80 dp tall field. The shorter title allows for this larger text size without risk of truncation.

Time Range Controls

The main content area opens with a horizontal row of controls for defining the time window rendered across all three charts. The layout and behavior of this row are identical to those on the Accelerometer screen.

A static "Von: " label precedes a compact `Button` (`button_x_von`) for selecting the start of the displayed time range. Its default text is empty (unlike the Accelerometer screen, which defaults to "01.01.2025 00:00"), meaning the displayed range label will be populated entirely at runtime. A "Bis: " label with a 10 dp start margin is followed by the matching end-range `Button` (`button_x_bis`), also with an empty default text. Both buttons are 30 dp tall with `@color/button` tinting and zero internal padding to keep them compact. Tapping either is expected to open a date-time picker dialog.

To the right, an `ImageButton` (`resetX`) bearing the reset icon (`outline_reset_focus_24`) restores the chart's time range to its default or full extent when tapped. A `Button` (`btn_x_10min`) labeled "10 Min", placed 60 dp to the right with `@color/header` tinting, provides a one-tap shortcut to zoom all three charts to the most recent ten minutes of gyroscope data.

Line Charts

Three equally sized `LineChart` views from the MPAndroidChart library are stacked vertically inside a weighted `LinearLayout` that occupies all remaining space between the time controls and the axis checkbox row. Each chart has 8 dp of margin on all sides.

- ****lineChartGyroX****: Plots the angular velocity measured along the X axis over the selected time range.

- ****lineChartGyroY****: Plots the angular velocity along the Y axis.

- ****lineChartGyroZ****: Plots the angular velocity along the Z axis.

Angular velocity is typically expressed in radians per second (rad/s). All three charts share the same time window set by the Von/Bis controls and the "10 Min" shortcut, so any time range adjustment is reflected across all three simultaneously. As with the Accelerometer charts, standard MPAndroidChart gestures are supported: pinch-to-zoom, drag to pan along the time axis, and tap to highlight individual data points.

Axis Visibility Checkboxes

At the bottom of the content area, a horizontal row of three equally weighted `CheckBox` controls allows the user to show or hide each individual axis chart:

- ****cbChartX**** – labeled "X axis", checked by default.

- ****cbChartY**** – labeled "Y axis", checked by default.

- ****cbChartZ**** – labeled "Z axis", checked by default.

All three are rendered in `@color/text` and are checked on launch, meaning all three charts are visible by default. Unchecking a box hides the corresponding chart, allowing the user to isolate one or two axes for a less cluttered view.

Footer Navigation Bar

The footer is a fixed-height (80 dp) horizontal `LinearLayout` (`footerBar`) with `@color/header` background. It provides left-right navigation between the sensor detail screens and contains four equally weighted elements.

On the left, a `TextView` (`accelDescription`) displays "Acc" at 20 sp, labeling the previous screen as the Accelerometer activity. Next to it, a `Button` (`btnPrevAccel`) labeled with a left arrow navigates to the Accelerometer chart screen when tapped. On the right side, a `Button` (`btnNextMagnet`) labeled with a right arrow navigates forward to the Magnetometer chart screen. A `TextView` (`btnGyro`) on the far right displays "Mag" at 20 sp, labeling the next destination as the Magnetometer screen.

This confirms the Gyroscope activity's position as the central screen in the three-screen sensor chart sequence: Accelerometer ← Gyroscope → Magnetometer. The navigation pattern is consistent with the footer of the Accelerometer screen, allowing the user to move freely between sensor detail views without returning to the main activity.

Note: The right-side footer label uses the view ID `btnGyro` despite displaying the text "Mag" and pointing to the Magnetometer screen. This appears to be a naming inconsistency in the source layout and does not affect runtime behavior.

 # Settings Activity (Einstellungen) – Layout Description

Overview

The Settings activity provides the user with controls for configuring application-wide preferences across four categories: language selection, data smoothing via the Douglas-Peucker algorithm, network and server connectivity, and push notifications. It is reachable from the main activity by tapping the settings gear icon in the header. The root layout is a vertical `LinearLayout` (`settings`) filling the entire display against a `@color/background` base. It is divided into two zones: a fixed header bar at the top, and a full-height `ScrollView` below it that contains all settings sections. Because the settings content may exceed the visible screen height, the scroll view ensures all options remain accessible on smaller devices without truncation.

Header Bar

The header is a horizontal `LinearLayout` with `@color/header` background, following the same pattern as the other secondary screens in the application. It contains two elements rather than the usual three.

On the left, an `ImageButton` (`home_button`) displays the home icon (`baseline_home_24`) with the content description "Zur Startseite". Tapping it returns the user to the main activity. In the center, a bold `TextView` (`headerText`) displays the screen title drawn from `@string/settings_title` at 28 sp, centered vertically and horizontally within an 80 dp tall field. A right margin of 40 dp is applied to the title to compensate for the absence of a right-side button, keeping the title visually centered relative to the full screen width rather than shifted left.

Scrollable Content Area

All settings sections are contained within a `ScrollView` that occupies all remaining vertical space via `layout_weight="1"`. Inside it, a vertical `LinearLayout` with 10 dp of margin holds four settings blocks, each separated from the next by a thin 1 dp horizontal divider line styled with `@color/button`.

Language Section

The first section is headed by a bold `TextView` at 20 sp drawing its label from `@string/settings_language_title`, with a 10 dp bottom margin. Below it, a `RadioGroup` (`rg_language`) presents three mutually exclusive language options in a vertical list:

- **rb_lang_de**: German, label from `@string/settings_lang_de`.

- **rb_lang_en**: English, label from `@string/settings_lang_en`.

- **rb_lang_zh**: Chinese, label from `@string/settings_lang_zh`.

Each radio button is rendered at 16 sp with 4 dp of vertical padding for comfortable tap targets. Selecting one option automatically deselects the others, as is standard `RadioGroup` behavior. The chosen language is expected to be applied app-wide, affecting all UI string resources.

Douglas-Peucker Section

The second section is headed by a bold 20 sp `TextView` labeled from `@string/settings_dp_title`. Below it, a descriptive `TextView` at 14 sp and 70% opacity, drawn from `@string/settings_dp_desc`, explains the purpose of the feature before the controls appear.

A `SwitchMaterial` (`switch_dp_active`) labeled from `@string/settings_dp_enable` at 18 sp allows the user to enable or disable the Douglas-Peucker algorithm entirely. This algorithm is a data reduction technique that simplifies sensor data series by removing points that fall within a configurable tolerance threshold, reducing the volume of data transmitted or stored without significantly distorting the signal shape.

When enabled, the epsilon (tolerance) parameter becomes relevant. A `TextView` (`tv_epsilon_value`) at 16 sp displays the current epsilon value label from `@string/settings_dp_epsilon_label`, updated dynamically as the user adjusts the control below it. A `SeekBar` (`seekbar_epsilon`) with a range of 0 to 100 and a default progress of 10 lets the user drag to set the desired epsilon value. A higher epsilon results in more aggressive data simplification; a lower value preserves more detail. A small italic hint `TextView` at 12 sp and 50% opacity, drawn from `@string/settings_dp_hint`, provides additional guidance on choosing an appropriate value.

Network & Server Section

The third section carries the hardcoded German title "Netzwerk & Server" at 20 sp bold. It contains two configurable items.

The first is a horizontal row pairing a `SwitchMaterial` (`switch_server_data`) labeled "Server-Datenabruf erlauben" (German for "Allow server data retrieval") at 18 sp with an `ImageButton`

(`btn_info_server_data`) showing an info icon (`baseline_info_outline_24`). Tapping the info button is expected to display an explanation of what enabling server communication entails. A short descriptive `TextView` at 14 sp and 70% opacity below the row reads: "Ermöglicht der App, mit dem Server zu kommunizieren" ("Allows the app to communicate with the server").

The second item is an `EditText` (`et_mqtt_broker_url`) for entering the MQTT broker address. It is labeled by a preceding `TextView` reading "Broker URL / IP-Adresse" at 18 sp and displays the placeholder hint "tcp://192.168.178.80:1883", indicating the expected input format of a full TCP URI including protocol, IP address, and port number. The input type is set to `textUri`, which surfaces a URL-friendly keyboard on most devices. The field's underline is tinted with `@color/text`. A descriptive `TextView` below it at 14 sp and 70% opacity clarifies that both local and cloud-based MQTT brokers are supported.

Notifications Section

The fourth and final section carries the hardcoded German title "Benachrichtigungen" ("Notifications") at 20 sp bold. It contains a single `SwitchMaterial` (`switch_push_notifications`) labeled "Push-Benachrichtigungen aktivieren" ("Activate push notifications") at 18 sp. A descriptive `TextView` below it at 14 sp and 70% opacity explains that enabling this option causes the system to send a push notification whenever one of the user's configured sensor rules is triggered. This ties the notification system to the event and rule logic defined elsewhere in the application.